LAPORAN TUGAS AKADEMIK

Pertemuan 5_. Function Perhitungan Pajak

Disusun Untuk Memenuhi Tugas Mata Kuliah Prak. Rekayasa &Arsitektur Basis Data

Dosen Pengampu : Cecep Kurnia Sastradipraja, S.Kom., M.Kom.



Di Susun Oleh :

Rahma Aulia Gantna 2507034

JURUSAN ILMU KOMPUTER

PROGRAM STUDI SISTEM INFORMASI

INSTITUT TEKNOLOGI

2026

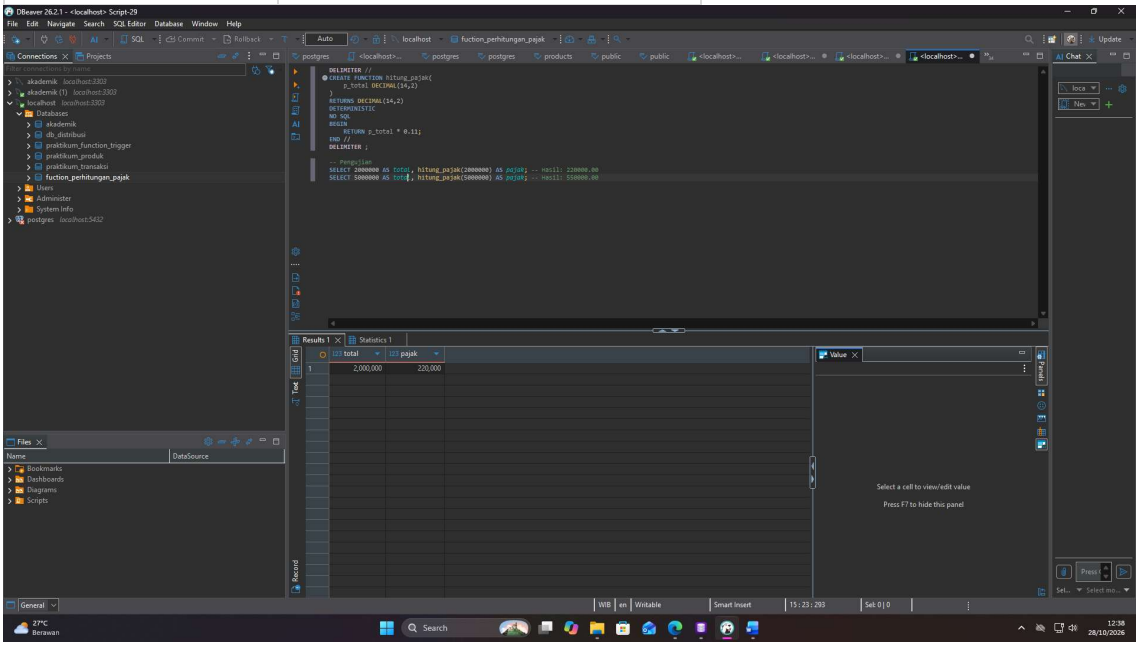
Tugas 1. Function Perhitungan Pajak

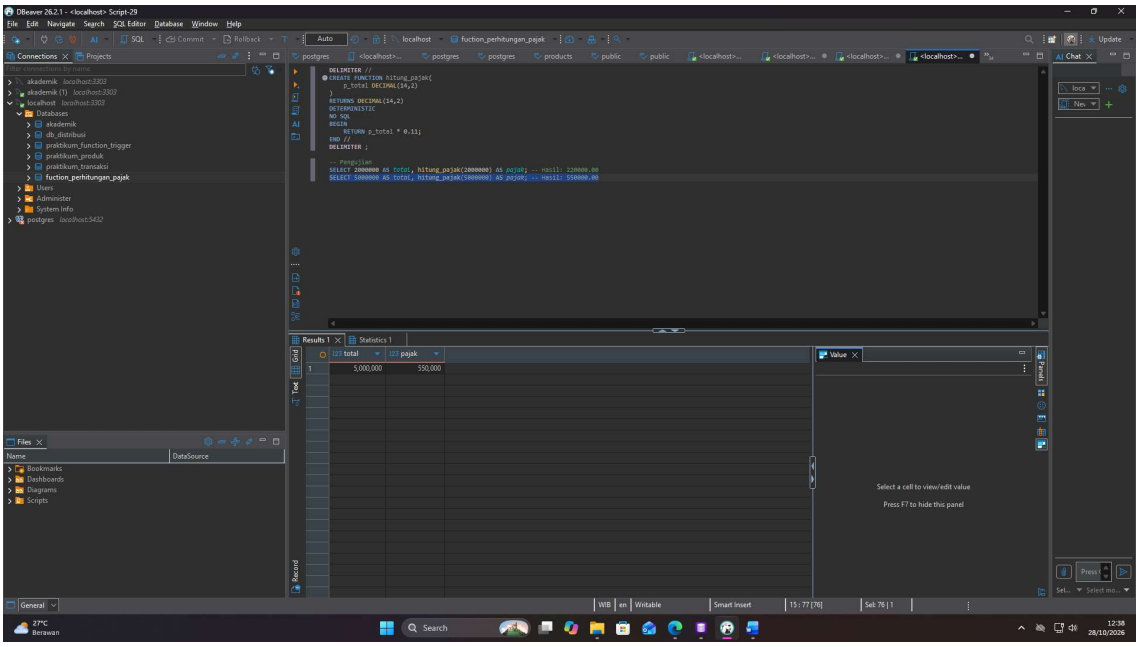
Bekal: Percobaan function hitung_total dan hitung_diskon.

Buat function hitung_pajak yang menerima total transaksi dan mengembalikan pajak sebesar 11%.

Uji menggunakan data berikut.

Total transaksi	**Pajak yang diharapkan**
Rp2.000.000	Rp220.000
Rp5.000.000	Rp550.000





Tugas 2. Function Total Akhir

Bekal: Percobaan function diskon dan penggabungan function.

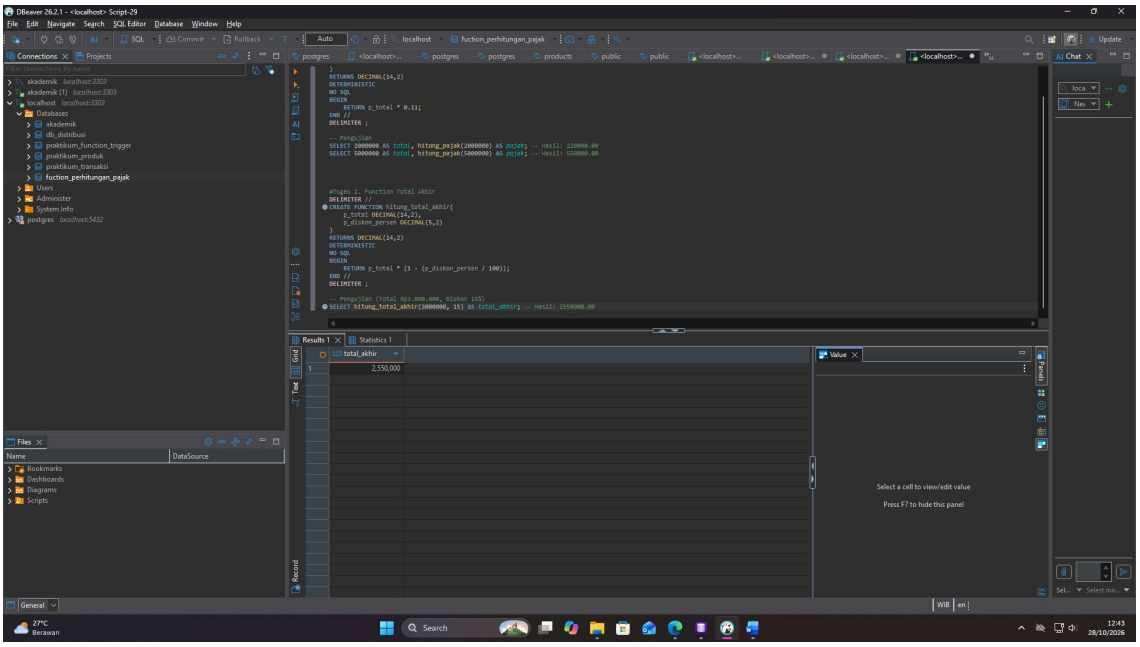
Buat function hitung_total_akhir yang menerima total transaksi dan persentase diskon.

Rumus:

$$\text{Total Akhir} = \text{Total Transaksi} \times \left(1 - \frac{\text{Diskon}}{100}\right)$$

Uji dengan total transaksi Rp3.000.000 dan diskon 15%.

Target hasil: Rp2.550.000.



Tugas 3. Trigger Pencatatan Perubahan Stok

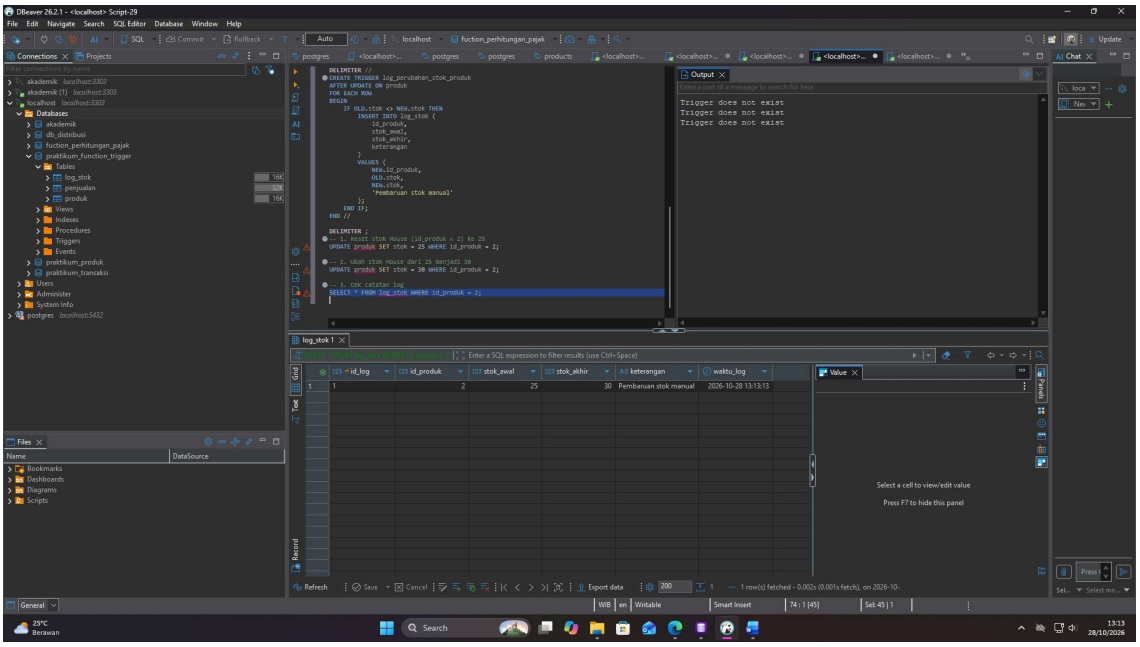
Bekal: Trigger AFTER INSERT, tabel log_stok, serta konsep OLD dan NEW.

Buat trigger AFTER UPDATE pada tabel produk untuk mencatat perubahan stok.

Ketentuan:

1. Catat stok sebelum perubahan menggunakan OLD.stok.
2. Catat stok setelah perubahan menggunakan NEW.stok.
3. Simpan ID produk, stok awal, stok akhir, keterangan, dan waktu perubahan.
4. Uji perubahan stok Mouse dari 25 menjadi 30 unit.

Jika stok telah berubah akibat praktikum sebelumnya, atur kembali stok ke 25 sebelum pengujian dan pastikan tidak ada transaksi pengujian yang mengganggu hasil.



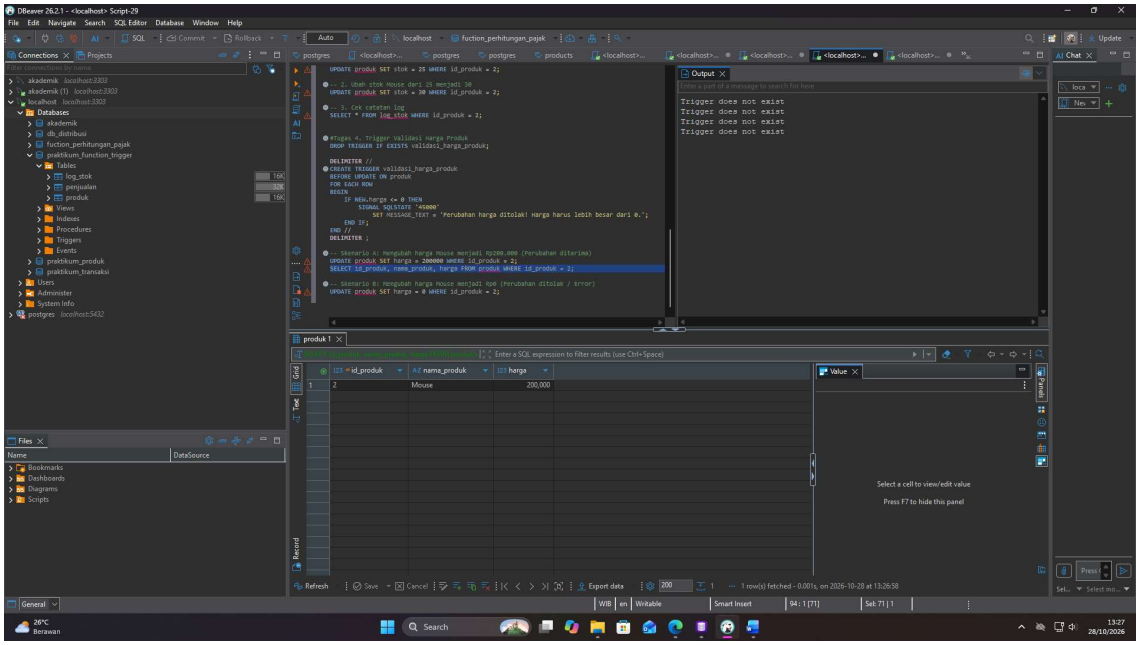
Tugas 4. Trigger Validasi Harga Produk

Bekal: Trigger BEFORE INSERT dan penggunaan SIGNAL SQLSTATE '45000'.

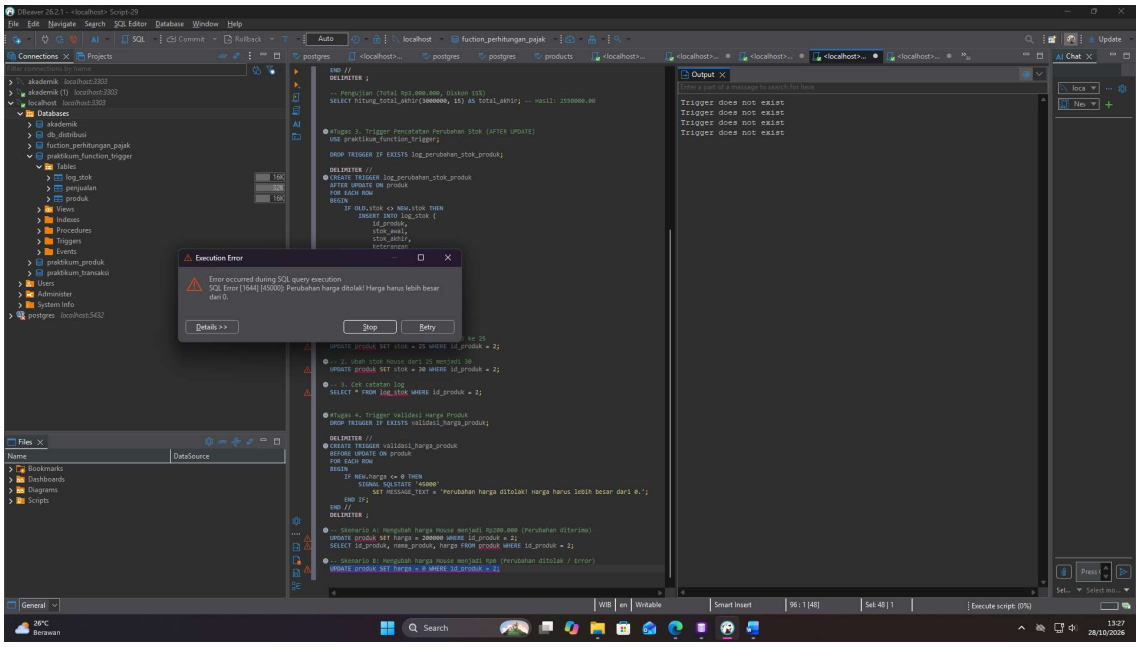
Buat trigger BEFORE UPDATE pada tabel produk untuk menolak perubahan harga menjadi nol atau negatif.

Uji dua skenario:

- Mengubah harga Mouse menjadi Rp200.000: perubahan diterima.



- Mengubah harga Mouse menjadi Rp0: perubahan ditolak.

- 

  ellor karena di tolak

Tugas 5. Analisis Integrasi Function dan Trigger

Jawab pertanyaan berikut berdasarkan hasil implementasi:

1. Apa perbedaan function hitung_total dengan trigger validasi_penjualan?
2. Apa dampaknya jika trigger pembaruan stok tidak dibuat?
3. Mengapa perubahan stok perlu dicatat dalam tabel log?
4. Bagaimana function dan trigger membantu menjaga integritas data?
5. Apa yang perlu diperhatikan ketika dua transaksi mencoba membeli produk dengan stok terbatas secara bersamaan?

jawaban

1. **Perbedaan Stored Function hitung_total dengan Database Trigger validasi_penjualan:**
   Stored Function (seperti hitung_total) bertindak sebagai kalkulator pasif yang menerima input nilai, melakukan kalkulasi matematika, dan mengembalikan hasil (*return value*) tanpa memodifikasi atau memeriksa aturan database secara langsung. Sedangkan *Database Trigger* (seperti validasi_penjualan) bertindak sebagai agen keamanan aktif yang terikat pada event DML (misal BEFORE INSERT). Trigger secara otomatis memutus dan membatalkan transaksi jika ada pelanggaran logika bisnis menggunakan perintah SIGNAL SQLSTATE '45000'.

2. **Dampak Jika Trigger Pembaruan Stok Tidak Dibuat:**
   Setiap transaksi penjualan yang tersimpan pada tabel penjualan tidak akan meng-update atau mengurangi angka persediaan pada tabel produk secara otomatis. Hal ini mengakibatkan *inconsistencies* (ketidaksesuaian data) antara catatan transaksi dengan sisa stok fisik di gudang, serta meningkatkan risiko human error jika pembaruan dilakukan secara manual.

3. **Alasan Perubahan Stok Perlu Dicatat dalam Tabel log_stok:**
   Pencatatan riwayat pada tabel log berguna sebagai *audit trail* (rekam jejak digital). Catatan ini penting untuk transparansi, melacak arus keluar-masuk persediaan barang, memantau riwayat pengubahan harga/stok oleh admin, serta mendeteksi indikasi fraud (kecurangan) apabila terjadi selisih stok.

4. **Peran Function dan Trigger dalam Menjaga Integritas Data:**
   Function memastikan seluruh perhitungan rumus bisnis seragam di seluruh aplikasi. Trigger menegakkan *domain integrity* dan *referential integrity* secara ketat langsung dari DBMS (misalnya menolak stok minus atau harga Rp0) tanpa bergantung pada pengkodean di aplikasi frontend/backend.

5. **Penanganan Dua Transaksi Bersamaan (*Concurrency/Race Condition*):**
   Diperlukan penguncian baris data (*Row-Level Locking*) dengan menyisipkan perintah FOR UPDATE pada query pengecekan stok di dalam trigger (seperti pada Percobaan 3.1). Mekanisme ini memastikan transaksi pertama mengunci data produk hingga prosesnya selesai (*commit/rollback*), sehingga transaksi kedua wajib antre dan mencegah stok menjadi minus akibat persaingan akses serentak.

6. Kesimpulan

Penggunaan **Stored Function** dan **Database Trigger** pada MySQL terbukti meningkatkan efisiensi komputasi, menjaga konsistensi data secara *real-time*, dan mengamankan basis data toko elektronik dari input data yang tidak valid. Penggabungan fungsi komputasi dan aturan trigger menjamin keandalan sistem basis data relasional.

7. Daftar Pustaka

1. Connolly, T., & Begg, C. (2015). *Database Systems: A Practical Approach to Design, Implementation, and Management* (6th ed.). Pearson.
2. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.